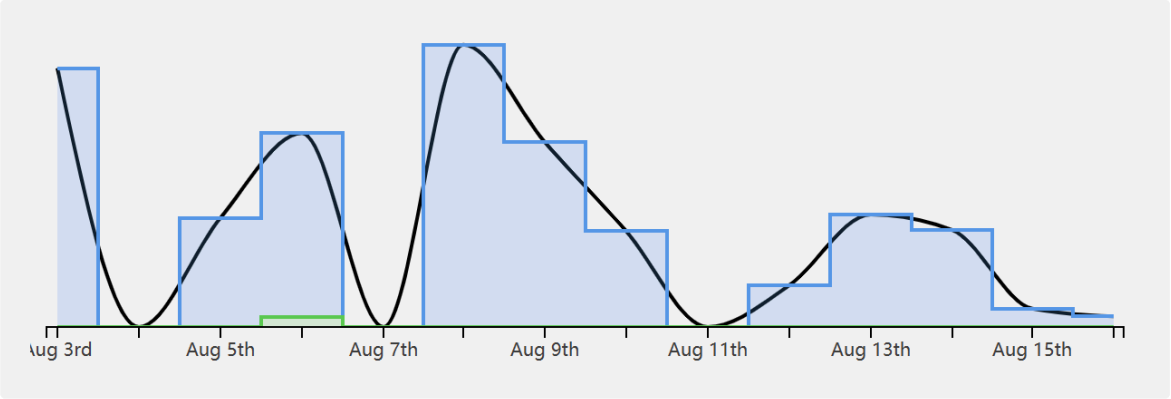
CS61B---Gitlet之我见

CS61B的课程项目有一大特点，便是**循序渐进**——当我刚刚接受完Java基本语法的熏陶之时，Project1的双端队列于我而言尚有一定难度。在文档中求索，在代码中前行。磕磕绊绊写完proj1的时候，鼠鼠内心不由得感到一丝恐慌——听说后面的Gitlet是个不得了的大家伙，而这先锋兵将proj1已让我汗流浃背(事后诸葛亮来看，应该是Java速通太快的缘故)！怀揣着这样忐忑又略显激动的心情，在扎扎实实啃完所有课程以及lab之后，我敲开了Gitlet的大门——

站在已经写完Gitlet的视角来看，它的特点并**不是难度高**，而是**文档冗长**以及**开发要求众多**。2W字的文档足以好好磨砺你的英文阅读能力。(或许这就是贴近实际开发过程的体现么，hhh)最后加上EC部分，总计用时大概是25h，不算EC是19h，如下：

(这段时间在学球，所以战线拉得很长，大概每天只能写2-3h。有时候教练不在，狂写一天，爽哉)

25 hrs 14 mins over the Last 14 Days.



我自己的实现：Gitlet

证明我写完了EC：

Project 2: Gitlet

● Ungraded

Autograder Score

1696.0 / 1600.0

Passed Tests

Velocity Limiting (0/0)

File Checking (0/0)

Compilation (0/0)

Style (0/0)

01) init (14.815/14.815)

02) basic-checkout (14.815/14.815)

03) basic-log (14.815/14.815)

04) prev-checkout (14.815/14.815)

10) init-err (14.815/14.815)

11) basic-status (14.815/14.815)

12) add-status (14.815/14.815)

13) remove-status (29.63/29.63)

14) add-remove-status (29.63/29.63)

15) remove-add-status (29.63/29.63)

16) empty-commit-err (29.63/29.63)

17) empty-commit-message-err (29.63/29.63)

18) nop-add (29.63/29.63)

19) add-missing-err (29.63/29.63)

```
20) status-after-commit (29.63/29.63)
21) nop-remove-err (29.63/29.63)
22) remove-deleted-file (29.63/29.63)
23) global-log (29.63/29.63)
24) global-log-prev (59.259/59.259)
```

```
101) ec-untracked (32/32)
102) ec-remote-fetch-push (24/24)
103) ec-remote-fetch-pull (24/24)
104) ec-bad-remotes-err (16/16)
```

下面是我自己的一些总结，建议，部分命令的思路以及它带给我的收获：

对后来人的建议

- 在开始写代码之前，**尽量**看完整个项目文档，在脑海中有一个大致完整的构思框架。这样可以避免一直写一直改之前的窘境，甚至会面临全部推倒重写！
- 良好的代码规范！给所有变量命名必须讲究到位，增加代码可读性，不然睡一觉起来就不知道自己干了啥了
- 多多写**注释**！这并不是浪费时间的习惯，相反，到位明了的注释在你debug时将会有良好的引导功能。很多时候我们的bug并不是大的逻辑有问题，而是就是在小的地方出了谬误，例如条件判断时的"!"，忘记一些必备基础操作等等，这种bug才是最难de的！
- **Commit，Blob，Stage，Branch，Remote**这些基础概念务必理解到位，**/.gitlet**仓库的结构务必安排好！
- Utils里的辅助方法**不是一成不变**的，你可以自己加一些有助于自己开发的函数进去！

Bug重灾区&&小技巧

- 忘记保存文件
- 忘记修改HEAD
- 忘记调整BRANCH
- 忘记add文件到Stage
- 生成id的时候不考虑实际因素(卖个关子)
- 忘记Java中对象指针值传递的本质(即浅拷贝)
- EC时忘记修改CWD
- 新建文件和目录可以在Utils里新加函数，避免反复写try-catch
- Commit的时间戳需要修改格式，记得写辅助方法
- EC后你的branches文件夹可能包含子文件夹，这时候可以修改那个读文件的辅助函数，保证status命令不会寄

- 代码复用，事实上reset，fetch，push，pull这四个命令几乎不需要怎么写，逻辑写出来，然后发现可以代码复用，就问你爽不爽吧。这也是之前说要**考虑清楚再动笔开始写代码**的原因所在

总体思路

/.gitlet仓库安排

根据文档要求，gitlet只处理平面文件夹里面的内容，不考虑递归处理子文件夹，那么可以讲Blob和Commit直接“平摊”存储在对应文件夹下。Stage，Branch，Remote分别存储在对应目录。这样的安排或许不是最佳，但是思路理解上还是较为中规中矩的，不是太难。

```
*  .gitlet/
*      - objects/
*          - commits/
*              - ...(files of commits)
*          - blobs/
*              - ...(files of blobs)
*      - branches/
*          - master
*          - ...(other branches)
*      - HEAD
*      - BRANCH
*      - Stages/
*          - addStage
*          - removeStage
```

其中HEAD和BRANCH是储存当前提交id和分支name的文件，随用随取，随取随读，随读随改，随改随存

类的分配

我特别喜欢写类，某种意义上写一个好的类也是代码复用的体现所在，所以有的地方或许可以用一个列表直接拉过去，但是我还是写了相应的类便于理解操作

- `Main`：主程序所在地，处理输入命令参数
- `Repository`：大部分代码所在地，实际编写处理命令的方法的地方
- `Commit`：提交的类，包含id，文件路径与BlobId的映射，Blob的列表，Message，Time
- `Blob`：文件的类，包含文件名字，内容，id
- `Stage`：缓存区的类，直接一个BlobId和文件路径的映射就完事，再编写一些辅助方法
- `Branch`：分支的类，包含分支名以及存储的CommitId
- `Remote`：远程的类，这个类其实可以不要的，但是我强迫症上来了()。包含远程名字和仓库路径

Persistence

对所有需要持久化的类写一个`saveToFile()`方法就完事，别忘了用就行

(不会有人还没写Lab 6吧，不会吧不会吧)

必做部分命令思路

太简单的就不写了罢……

init

虽然说测试中只检测了你的/.gitlet目录，但是还是有更多工作要做，按照文档一步步来

- 检测文件夹是否存在，存在就退出
- 创建一个新的分支master，这也是初始默认分支
- 创建初始提交，让master指向它
- 创建各级目录
- 创建两个空的缓存区

你会发现有的东西涉及到后面的命令，这时候可以把写不了的部分打一个TODO，后面来补

推荐用时：1-2h，加上理解以及写各类可能会到4h

后面的每个命令都需要**检查文件夹是否存在**了，不能不存在

add

虽然说只需要将对应文件添加到缓存区，但是仍然要考虑其他的情况

- 先检查该检查的
- 如果文件在removeStage里，移出(涉及到rm命令，可以留白)
- 创建新的Blob(这些对应的类你应该已经写好十之八九了)
- 添加到addStage
- 不要忘了保存

推荐用时：1h

commit

第一个坎，建议写了rm后再来写，因为rm可以看作是add的逆命令，而commit和这两个都有关系

- 构造函数可以写两个，一个用于普通commit，一个是初始commit专用
- id生成记得用`toString()`一下类里的其他成员
- 先检查
- 生成新的commit，这时候它的内容除了时间和它的parent是一样的
- 根据两个stage修改内容
- 修改HEAD与当前分支的内容
- 保存

推荐用时：2h

rm

有了写add的经验写这个应该比较快，注意分类讨论：

- 在addStage里，删除之
- 不在addStage里，但在当前commit里(即最新commit)，加入removeStage，commit的时候删除
- 是上面第二种情况的话，CWD中不能存在该文件，即存在就删掉

推荐用时：1h

log && global-log

- 前者DFS，后者直接用plain那啥的辅助函数
- 可以考虑给Commit类写一个`print()`的辅助方法

find

这个简单，正式git都没有，考你对自己写的类的掌握情况呢

推荐用时：10min

status

这个分类有点多，最后两个还是ec的部分，一步一步来即可

- branch：直接读目录里的文件就行
- 两个stage：把stage的文件读出来就行，也可为Stage类写一个`print()`的辅助方法
- 修改未保存：按照文档分的四类一步步写即可，注意blobId即可代表两个文件是否相同
- 未跟踪：读CWD文件，一个一个判断即可

推荐用时：20min(无EC)，1h(含EC)

checkout

三个类型一步步写，其实这里建议在两个branch命令之后写

- 检出文件：建议写一个辅助方法，后面会用到
- 检出某个commit里的某文件：同上
- 检出某个分支：再写一个辅助方法，完成这里这个命令的同时也为后面reset服务

推荐用时：4h(debug麻了)

branch && rm-branch

虽然branch的概念很高大上，复杂，但是不妨碍这两个命令很简单啊！稍加思考即可发现：branch功能本身的实现是在merge，checkout一系列命令的加持下才成型的。对于这两个命令，我们只需要做该做的事即可

- branch：创建这么一个对象，保存
- rm-branch：找到这个文件，删掉它
- 记得检查！
- 没了

建议用时：30min

reset

提示：用checkout的某类情况的辅助方法，实际上这个reset也是真实git的checkout的功能之一(但是不会头分离，最接近reset --HARD)

建议用时：10min

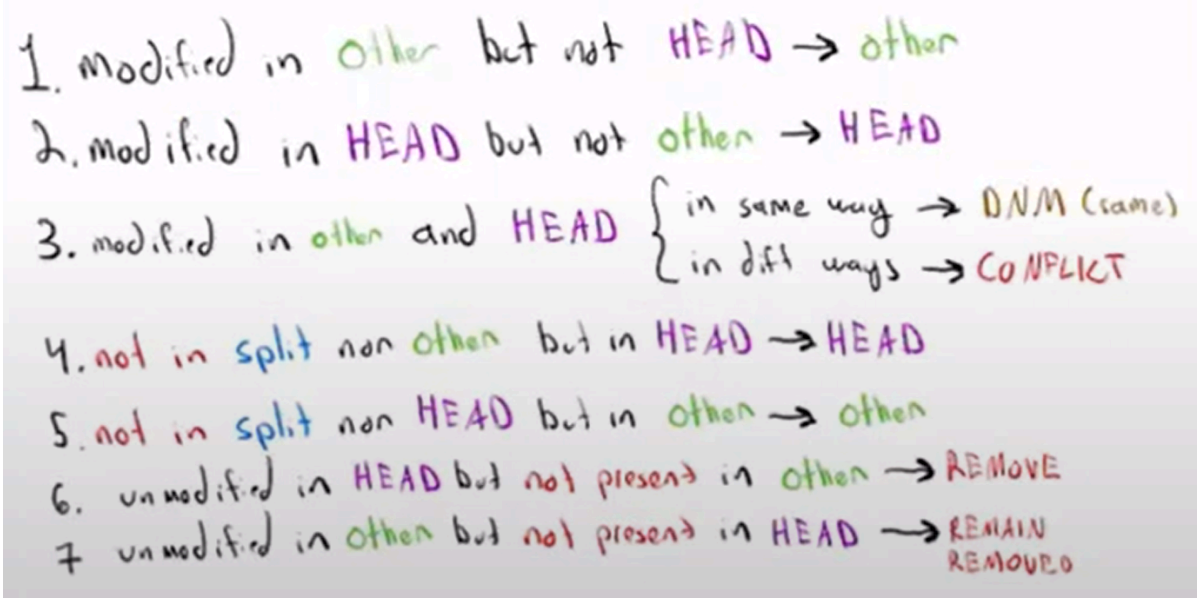
merge

必做部分的最大boss，推荐先观看视频，深入了解merge的对每个文件处理的八个情况分别是什么，再拟出自己的思路，一步一步写，并且随时debug，这样可以减轻自己的痛苦时间()

我的总体思路如下：

- 检查
- 处理easy cases
- 用BFS获取splitCommit
- 现在手上有三个commit：分裂点，待合并的branch指向点，当前点
- 获取其中所有文件，一一处理，这一步我的选择是返回一个result的map，因为我还没检查，要检查后才能操作
- 检查result是否合法
- 检查完毕，根据result的指示处理文件(考虑用一个flag记录是否发生conflict)
- 处理完毕，创建merge commit
- 根据处理结果(两个stage)修改commit
- 保存之，若conflict则打印相关信息
- 清空缓存，设置HEAD，BRANCH文件，欢呼！

对于文件处理，牢TA的分类：



我来给它升个级：

- 对于每一个文件，它在splitcommit，branchcommit，currentcommit均有相应的形态(**不存在也是形态之一**)
- 若三者皆不相同，就是conflict！
- 若三者皆相同——那讨论个毛线！
- 若两两相同，进一步分类：
  - s与另两个之一相同，保留第三者
  - s与另两个都不同，但另两个相同——保留current即可(即不做操作)

一下子就只有三类了，明了了许多对不对

推荐用时：6h，但是debug无上限(我自己就几个小bug，比如忘记写！，忘记add，却debug了五六个小时)

恭喜！1600分到手！

选做部分命令思路

总体不难，因为有大量代码复用，注意规划好类与文件夹分区即可

总体思路：每个remote是一个对象，里面包含了remote的名字和远程仓库路径。编写一个改变当前CWD的辅助函数用于操作fetch，push与pull。似曾相识的地方就代码复用，必要时小改之前的命令用于抽出辅助函数复用。

具体命令思路：

add-remote && rm-remote

类比branch那边，虽然remote是一个大的复杂的概念，但是这俩命令不难呀，按自己的思路写好就行，我的思路：

- add-remote：创建remote对象，保存(文档要求不用检查合法性)
- rm-remote：找到那个remote文件，删掉！(记得先检查这个remote是否存在)

推荐用时：30min

fetch

我的思路里，核心在于改变CWD以及复制commit和blob们(复制commit时我出了超级逆天爆冷大bug，下面详谈)

fetch的要求是将远程的分支拽过来但是不合并，合并是pull的事情，咱fetch不用管，自然有了如下思路：

- 检查文件夹以及参数是否合法
- 拿到远程分支，用BFS取到它的历史(代码复用merge的部分)
- 回到本地工作目录
- 开始复制！注意！复制的时候不能只傻傻地把文件拉过来，还要修改内部文件的路径(注意这里并不会影响id生成，思考这是如何实现的？)，不然你后面checkout的时候有好果汁吃，就是这里吃掉我2h的debug时间
- 修改本地远程跟踪分支，若不存在则创建，注意文件名不能有/，所以我直接创建的子文件夹
- 上述过程中注意随时切换CWD

推荐用时：3h

push

类比为fetch的反向特化版，因为它的条件要求更苛刻，并且复制那一部分的逻辑是一样的，思路如下：

- 先老规矩，检查
- BFS获取本地分支历史(和上面不一样吧，但是直接美美复用)
- 检查远程分支是否存在
    - 若是，获取那边的头部，检查是否在本地历史中
    - 若否，直接进入下一步

- 向远程仓库复制commit和blob
- 修改远程分支，不存在则创建，记得保存
- 没有啦

推荐用时：1h

pull

文档已经写好了：fetch + merge，两行代码搞定！

推荐用时：2 min 30 s (**只因**代码复用**太美**)

恭喜！完整版Gitlet已完成！

总结与收获

代码量的话，我的repo类是1200行，不算ec是900行。如果少啰嗦一点可以更节约

总体难度其实不大，主要是量确实大，以及起步很难，不愣个一天神是不行的

收获真的很大，最重要一点就是克服了之前就有的项目恐惧症——

- 总觉得61A的项目太小儿科，对自己的码力没有自信
- 面对大项目就发怵，心里没底
- 写了一堆代码后很害怕debug，不知道它会藏在哪里暗杀我

除此之外便是一次完整的超级mini版的开发体验，用git做版本控制工具写gitlet，真是奇妙的体验呀！

记录自身成长的同时，也希望能够帮助到你！谢谢！